

# 5G ve Yapay Zekâ ile Akıllı Yol Güvenliği Yarışması

## FTR Aşaması Teslim Dokümantasyonu

Değerli Yarışmacılar,

**5G & Yapay Zekâ ile Akıllı Yol Güvenliği Yarışması**'nda ilk aşama olan Ön Tasarım Raporu aşaması başarıyla tamamlanmış ve tanımladığımız senaryo için yarışmacıların model geliştirecekleri ikinci aşama olan Final Tasarım Raporu (FTR) sürecine adım atılmıştır.

Bu doküman, geliştireceğiniz yapay zekâ modellerinin çıkarım (inference) sonuçlarını üreteceği formatı ve değerlendirme scriptlerimizin kabul edeceği standart etiket değerlerini içermektedir. Değerlendirme süreçlerinin adil, hızlı ve hatasız tamamlanabilmesi için çıktılarınızın burada belirtilen formata ve etiket sınıflarına **birebir uyması zorunludur**. Küçük-büyük harf duyarlılığına, Türkçe karakter kısıtlamalarına ve JSON şema yapılarına azami önem göstermeniz gerekmektedir.

### 1. Senaryo Kapsamı ve Genel Kurallar

**Araç Sınırı:** Test videolarımızda aynı anda kadrja giren **tek bir ana araç** bulunacaktır. Video akışı boyunca en az 1, en fazla ise belirsiz sayıda araç geçişi veya takibi gerçekleştirilebilir.

**Türkçe Karakter Kısıtlaması:** Otomasyon scriptleri ile uyumluluk açısından tüm kategori adları ve sınıf etiketleri **Türkçe karakter içermeyen (ASCII-safe) ve küçük harfli** standart metinler olmalıdır.

**Hatasız Eşleşme:** Çıktıların otomatik olarak taranıp puanlanabilmesi için JSON anahtarları (keys) ve etiket değerleri (values) aşağıdaki tablolarda tanımlanan değerlerle tam olarak eşleşmelidir.

### 2. Beklenen Değerler ve Sınıf Etiketleri

Tablo 1: Araç Bilgisi Beklenen Değerler (arac\_bilgisi)

Sistemde tespit edilen araca ait genel bilgilerin çıkartılması beklenmektedir. Aşağıdaki alanlar ve değer sınıfları kullanılmalıdır:

| Alan (Field)     | Beklenen Değerler / Format                                                                                                                                                                                         | Açıklama                                                                                                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| tip              | sedan, suv, hatchback, pickup, minibus, panelvan, kamyon                                                                                                                                                           | Araçın gövde tipi sınıflandırması.                                                                       |
| plaka            | Türkiye plaka standartlarına uygun formatta olmalıdır. <b>Örneğin:</b> 34ABC123<br><b>Regex:</b> <code>^(0[1-9] [1-7][0-9] 8[01])(\s?[a-zA-Z]\s?){4,5} (\s?[a-zA-Z]{2}\s?){3,4} (\s?[a-zA-Z]{3}\s?){2,3})\$</code> | Plaka üzerinde Türkçe karakterler veya boşluklar standart regex kontrolüne göre çözümlenmelidir.         |
| renk             | beyaz, siyah, gri, kırmızı, mavi, sarı, yeşil, turuncu, kahverengi                                                                                                                                                 | Araçın ana dış gövde rengi (Tamamı küçük harf ve ASCII karakterlerle).                                   |
| confidence_score | 0 ile 1 arasında ondalık sayı (float).                                                                                                                                                                             | Araç bilgisindeki <b>tip, plaka ve renk</b> tahminlerinin bütününe temsil eden genel bir doğruluk skoru. |

Tablo 2: Yol Güvenliği Tespitleri ve Aksiyonlar (tespitler)

Yol kenarı kameralarından elde edilen video akışlarında yol güvenliğini tehdit edebilecek sürücü aksiyonları, yolcular veya kabin içi nesnelerin tespiti için aşağıdaki etiketler tanımlanmıştır:

| Kategori             | Geçerli Etiketler                                                                                           | Açıklama                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Sürücü eylemi</b> | arkaya_bakma, esneme, sigara_icme, su_icme, telefonla_konusma, slalom, etrafa_bakinma, emniyet_kemer_ihlali | Sürücünün anlık dikkat dağınıklığı, yorgunluk veya kural ihlali durumlarını gösteren eylemler. |
| <b>nesneler</b>      | teknocan, bilgisayar                                                                                        | Araç içi kabinde veya yol üzerinde tespit edilen hedef nesneler.                               |
| <b>yolcular</b>      | arka_koltuk_1, arka_koltuk_2, on_koltuk                                                                     | Araç içinde bulunan yolcuların konum bazlı tespiti.                                            |

### 3. Güven Skoru (Confidence Score) ve Zamanlama Kuralları

**Güven Skoru Değer Aralığı:** Yapay zekâ modelinizin yaptığı sınıflandırma, nesne tespiti veya plaka tanıma adımlarının her biri için 0.0 (En düşük güven) ile 1.0 (Tam güven) arasında bir ondalık sayı (float) döndürülmelidir.

**Araç Tanıma için:** Tüm araç özelliklerinin (tip, plaka, renk) bütünü için tek bir ortak güven skoru belirlenmeli ve JSON çıktısında "confidence\_score" anahtarı altında iletilmelidir.

**Nesne/Aksiyon Tespiti için:** Tespit edilen her bir eylem, yolcu veya nesne için ayrı ayrı saniye bilgisi ve ilgili anlık "confidence\_score" değeri dönülmelidir.

### 4. Çıktı JSON Şemaları

Modellerinizin test videoları üzerinde yaptığı analizler sonucunda üreteceği çıktı aşağıda anlatılan biçimde, results.json adıyla /app/data/output/results.json yoluna kaydedilmelidir.

#### 4.1. Nesne ve Aksiyon Tanıma Çıktı Formatı

Sürücü eylemlerinin, kabin içi nesnelerin ve yolcuların zaman bazlı (zaman\_saniye) tespitlerini içeren dizi yapısıdır:

```
{
  "video_id": "video_001.mp4",
  "tespitler": [
    {
      "zaman_saniye": 14.5,
      "kategori": "sofor_eylemi",
      "etiket": "telefonla_konusma",
      "confidence_score": 0.89
    },
    {
      "zaman_saniye": 22.1,
      "kategori": "nesneler",
      "etiket": "teknocan",
      "confidence_score": 0.95
    },
    {
      "zaman_saniye": 45.8,
      "kategori": "yolcular",
      "etiket": "on_koltuk",
      "confidence_score": 0.91
    }
  ]
}
```

## 4.2. Araç Tanıma Çıktı Formatı

Görüntüdeki araca dair tip, plaka ve renk özetini içeren yapılandırılmış nesne yapısıdır:

```
{
  "video_id": "video_001.mp4",
  "arac_bilgisi": {
    "tip": "sedan",
    "plaka": "34ABC123",
    "renk": "beyaz",
    "confidence_score": 0.94
  }
}
```

## 4.3. Konsolide Çıktı Dosyası

Her video için yapılan tespitlerin sonunda bu results.json dosyasının oluşturulması beklenmektedir. Dosyada hem (4.1) başlığında tanımlanan tespitler listesi, hem de (4.2) başlığında tanımlanan arac\_bilgisi nesnesi yer almalıdır. Örneğin:

```
{
  "video_id": "video_001.mp4",
  "arac_bilgisi": {
    "tip": "sedan",
    "plaka": "34ABC123",
    "renk": "beyaz",
    "confidence_score": 0.94
  },
  "tespitler": [
    {
      "zaman_saniye": 14.5,
      "kategori": "sofor_eylemi",
      "etiket": "telefonla_konusma",
      "confidence_score": 0.89
    },
    {
      "zaman_saniye": 22.1,
      "kategori": "nesneler",
      "etiket": "teknocan",
      "confidence_score": 0.95
    }
  ]
}
```

## 5. En Sık Yapılan Hatalar ve Altın Kurallar

1. **İsimlendirme Hataları:** JSON çıktısında "confidence\_score" yerine "guven\_skoru" veya "score" gibi farklı anahtarlar kullanılması otomatik değerlendirme scriptinin hata almasına sebep olur. Lütfen JSON örneklerindeki anahtar isimlerini birebir kullanın.
2. **Karakter Kodlama:** Etiketlerde Türkçe karakter (ç, ğ, ı, ö, ş, ü) kullanılmamalıdır. Örneğin; kırmızı yerine kırmizi, şoför\_eylemi yerine sofor\_eylemi değerleri üretilmelidir.
3. **Plaka Formatı:** Çıktıda plaka bilgisi okunurken aradaki boşlukların regex kurallarına uygun şekilde normalize edilmesi veya doğrudan birleşik şekilde (34ABC123) yazılması gerekmektedir. Regex sınırları dışındaki plakalar "tespit edilemedi" olarak puanlanacaktır.
4. Gönderilen kodlar **hakem tarafından incelenecektir**. Submission içerisinde; ortam değişkeni, hostname, IP adresi veya dosya varlığı kontrolü gibi yöntemlerle **"değerlendirme ortamında mıyım?" tespiti yapan ve buna göre farklı davranış sergileyen yapılar** (örn. if/else, try/except

tabanlı manipölasyon) tespit edilmesi halinde ilgili takımın deęerlendirmesi **geersiz sayılacaktır**.

## 6. Dockerizasyon ve Deęerlendirme Sunucusu (VM) Özellikleri

Geliřtirdiđiniz projeler, otomatik deęerlendirme platformumuz tarafından izole bir Docker konteyneri olarak ayaęa kaldırılacak ve fiziksel NVIDIA Tesla T4 GPU barındıran sunucularda test edilecektir. Deęerlendirme sunucusunun donanım ve yazılım kısıtları řu řekildedir:

| Özellik                 | Kriter / Deęer                      |
|-------------------------|-------------------------------------|
| İřlemci Gücü (vCPU)     | 4.0 vCPU                            |
| Sistem Belleęi (RAM)    | 16 GB RAM                           |
| Grafik İřlemci (GPU)    | NVIDIA Tesla T4                     |
| Paylařımlı Bellek (SHM) | 2 GB                                |
| Temel İmaj (Base Image) | nvidia/cuda:12.1.0-base-ubuntu22.04 |
| İmaj Boyutu             | Maksimum 8 GB                       |
| Giriř Yolu (Path)       | /app/data/input/video.mp4           |
| Çıktı Yolu (Path)       | /app/data/output/results.json       |
| Model Aęırlıkları       | /app/models/                        |
| Maks. Çalışma Süresi    | 10 dakika                           |

**Önemli Not:** Dockerfile build edilirken geen süre, kodun çalışma süresine (Timeout) dahil edilmeyecektir.

## 7. Veri Seti, Eęitim ve Model Dayanıklılık Kořulları

- Aık Kaynaklı Veri Setleri:** Yarışmacılar modellerini eęitmek ve geliřtirmek amacıyla aık kaynaklı akademik veya genel veri setlerini kullanmakta tamamen özgürdür. Deęerlendirme komitesi tarafından örnek video görüntüleri haricinde ekstra bir eęitim veri seti paylařılmayacaktır.
- Model Gürbüzlüęü (Robustness):** Geliřtirilen modeller; farklı FPS (Kare hızı) deęerleri, deęiřken video çözünürlükleri, zorlu hava kořulları (yaęmur, sis vb.) ve farklı ıřık/parlama durumlarında kararlı çalışabilecek řekilde eęitilmelidir. Deęerlendirme videoları bu çeřitlilikleri barındıracaktır.
- Hata Yönetimi:** Kodunuzun girdi dizininde video bulamaması veya hasarlı bir video ile karřılařması durumunda ökmesini engellemek için gerekli try-except bloklarını eklemeniz önerilir.

## 8. Örnek Proje Yapısı ve Docker Dosyaları

Projenizi paketlerken Dockerfile dosyasının projenizin **en üst dizininde (root level)** bulunması zorunludur. docker run ... komutu çalıştırıldığında, programınız ekstra manuel adıma gerek duymadan, Dockerfile'daki CMD veya ENTRYPOINT aracılığıyla **otomatik olarak bařlamalıdır**.

Saęladığınız kaynak kodlar ve Dockerfile sayesinde, hakem sizin yerinize ařağıdakileri gerekleřtirir:

```
# 1) Dockerfile'ınızdan image'ı oluřturur
docker build -t teknofest/<projeniz>:latest .
```

```
# 2) Tesla T4 üzerinde çalıştırır; videoyu /app/data/input'a, çıktı klasörünü
/app/data/output'a baęlar
docker run --rm --gpus all \
-v <video-dosyası>:/app/data/input/video.mp4 \
-v <run-klasörü>:/app/data/output \
teknofest/<projeniz>:latest
```

### Örnek Dizin Yapısı:

```
./takim_adi/  
├── imaj.tar  
├── Dockerfile  
├── README.md  
├── requirements.txt  
├── main.py  
├── src/  
│   ├── predict.py  
│   └── utils.py  
└── weights/  
    └── best_model.pt
```

### Örnek Dockerfile:

```
# 1. Adım: Resmi CUDA temel imajını kullanıyoruz  
FROM nvidia/cuda:12.1.0-base-ubuntu22.04  
  
# Gerekli sistem paketlerini kuruyoruz  
RUN apt-get update && apt-get install -y \  
    python3 \  
    python3-pip \  
    ffmpeg \  
    libsm6 \  
    libxext6 \  
    && rm -rf /var/lib/apt/lists/*  
  
# Çalışma dizinini belirliyoruz  
WORKDIR /app  
  
# Gerekli klasör yapılarını oluşturuyoruz  
RUN mkdir -p /app/data/input /app/data/output /app/weights /app/src  
  
# Gereksinimler dosyasını kopyalıyoruz  
COPY requirements.txt .  
  
# Tüm bağımlılıkları build aşamasında kuruyoruz (Çalışma anında internet kapalı olacaktır)  
RUN pip3 install --no-cache-dir -r requirements.txt  
  
# Model ağırlıklarını weights/ klasöründen kopyalıyoruz  
COPY weights/ /app/weights/  
  
# Modüler kaynak kodlarımızı kopyalıyoruz  
# (imaj.tar dosyasının imaj boyutunu aşmaması için COPY . . yerine seçici COPY yapıyoruz)  
COPY src/ /app/src/  
COPY main.py .  
COPY README.md .  
  
# Konteyner ayağa kalktığında otomatik çalışacak komut  
CMD ["python3", "main.py"]
```

### Örnek main.py Giriş Noktası:

```
import os  
import json  
import sys  
  
# src klasöründeki modüllere erişim sağlamak için import yollarını ekliyoruz
```

```

sys.path.append(os.path.abspath(os.path.dirname(__file__)))

# src/predict.py içerisindeki ana çıkarım fonksiyonunu çağırıyoruz
from src.predict import run_inference

def main():
    input_path = "/app/data/input/video.mp4"
    output_path = "/app/data/output/results.json"
    weights_path = "/app/weights/best_model.pt"

    # Girdi kontrolü
    if not os.path.exists(input_path):
        print(f"Hata: Girdi videosu bulunamadı -> {input_path}")
        sys.exit(1)

    print("Yol Güvenliği Yapay Zeka Çıkarım İşlemi Başlatıldı...")
    print(f"Model Ağırlığı: {weights_path}")

    try:
        # Görsel analiz işlemini tetikliyoruz
        output_data = run_inference(input_path, weights_path)

        # Çıktı dizininin varlığını garanti altına alıyoruz
        os.makedirs(os.path.dirname(output_path), exist_ok=True)

        # Sonuçları standart JSON formatında diske yazıyoruz
        with open(output_path, "w", encoding="utf-8") as f:
            json.dump(output_data, f, ensure_ascii=False, indent=2)

        print(f"İşlem başarıyla tamamlandı. Çıktı kaydedildi: {output_path}")

    except Exception as e:
        print(f"Model çalıştırılırken bir hata ile karşılaşıldı: {str(e)}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

### Örnek src/predict.py Dosyası:

```

import os
from src.utils import preprocess_frame, format_output

def run_inference(video_path, weights_path):
    """
    Video akışını okuyup, model ağırlıkları ile çıkarım yapan ana fonksiyon.
    """
    print(f"[Inference] {video_path} dosyası analiz ediliyor...")

    # Burada openCV veya benzeri bir kütüphaneyle videoyu frame okuyup,
    # weights_path içerisindeki modelinizle tahminleme yapmanız gerekmektedir.
    # preprocess_frame(frame) kullanarak ön işlem yapabilirsiniz.

    # Simüle edilmiş tahmin verileri (Örnektir)
    raw_predictions = {
        "vehicle_type": "suv",
        "license_plate": "34TC2026",
        "color": "mavi",
        "confidence": 0.96,
        "events": [

```

```

        {"time": 12.4, "cat": "sofor_eylemi", "label": "esneme", "conf": 0.88},
        {"time": 35.1, "cat": "nesneler", "label": "bilgisayar", "conf": 0.92}
    ]
}

# Sonuçları yarışma standart çıktı şemasına uygun hale dönüştürüyoruz
return format_output(raw_predictions)

```

Örnek `src/utils.py` Dosyası:

```

def preprocess_frame(frame):
    """
    Görüntü ön işleme adımları (Boyutlandırma, normalizasyon vb.)
    """
    # Modelinize özel ön işleme kodları burada yer alacaktır.
    return frame

def format_output(predictions):
    """
    Yarışma standartlarına uygun JSON şemasını hazırlar.
    """
    return {
        "video_id": "video.mp4",
        "arac_bilgisi": {
            "tip": predictions["vehicle_type"],
            "plaka": predictions["license_plate"],
            "renk": predictions["color"],
            "confidence_score": predictions["confidence"]
        },
        "tespitler": [
            {
                "zaman_saniye": event["time"],
                "kategori": event["cat"],
                "etiket": event["label"],
                "confidence_score": event["conf"]
            }
            for event in predictions["events"]
        ]
    }

```

## 9. Teslim Öncesi Son Kontrol Listesi

| Kontrol Maddesi                                                                                       | Durum |
|-------------------------------------------------------------------------------------------------------|-------|
| Dockerfile dosyanız projenin en üst dizininde mi?                                                     | E/H   |
| Temel imajınız <code>nvidia/cuda:12.1.0-base-ubuntu22.04</code> olarak ayarlandı mı?                  | E/H   |
| Modeliniz GPU üzerinde (cuda) çalışacak şekilde konfigüre edildi mi?                                  | E/H   |
| Programınız çalışmaya başladığında <code>/app/data/input/video.mp4</code> yolundan videoyu okuyor mu? | E/H   |
| Tüm analiz sonuçlarınız <code>/app/data/output/results.json</code> yoluna yazılıyor mu?               | E/H   |
| Çıktı JSON'ındaki tüm etiketler ASCII karakterlere ve küçük harf kuralına uygun mu?                   | E/H   |
| <code>docker run ...</code> komutunu çalıştırıldığında kodunuz otomatik olarak başlıyor mu?           | E/H   |

**Tüm takımlarımıza model geliştirme süreçlerinde başarılar dileriz!**